

Photographer Booking

Internship Report Submitted to

AYYA NADAR JANAKI AMMAL COLLEGE, SIVAKASI.

affiliated to the **Madurai Kamaraj University, Madurai**, in partial fulfilment of the requirements for the award of the Degree Programme of

MASTER OF COMPUTER APPLICATIONS

Submitted by

R. PANDISELVAN

(Register No: 25PCA140)

Internship offered by

Abservetech Private Limited,

Madurai

Mentored by

Ms P. BARKAVI MCA

Assistant Professor



POST GRADUATE DEPARTMENT OF COMPUTER APPLICATIONS

AYYA NADAR JANAKI AMMAL COLLEGE

(Autonomous, affiliated to **Madurai Kamaraj University**, Re-accredited (4th Cycle) with 'A⁺' Grade (CGPA of 3.48 out of 4) by NAAC, Recognized as College of Excellence and Mentor Institution by UGC, Star College by DBT and Ranked 72nd at National Level in NIRF 2025 and DST-FIST (2023) Supported & An ISO 9001:2015 & ISO 21001:2018 Certified Institution)

SIVAKASI – 626 124.

July - 2026

Dr. R. VENGATESHKUMAR, M.C.A., M.Phil., Ph.D.

Director,

Post Graduate Department of Computer Applications,

Ayya Nadar Janaki Ammal College (Autonomous),

Sivakasi.

CERTIFICATE

This is to certify that this “**Photographer Booking**” being submitted by **R.PANDISELVAN (25PCA140)**, final year student of Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by him/her under the guidance of **Mrs P. BARKAVI MCA.**, Assistant Professor, Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi.

Place: Sivakasi,

Date:

Signature of the Director

(Dr. R. VENGATESHKUMAR)

Ms P. BARKAVI MCA,
Assistant Professor,
Post Graduate Department of Computer Applications,
Ayya Nadar Janaki Ammal College (Autonomous),
Sivakasi.

CERTIFICATE

This is to certify that this **“Photographer Booking”** being submitted by **R.PANDISELVAN (25PCA140)**, final year student of Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, is a *bonafide* record of work carried out by him/her under my guidance and supervision.

It is further certified that to the best of my knowledge, this Internship report or any part thereof has not been submitted in this College or elsewhere for the award of any other degree or diploma.

Place : Sivakasi,

Date:

Signature of the Guide
(Mrs P. BARKAVI)

R.PANDISELVAN,
25PCA140,
II MCA,
Post Graduate Department of Computer Applications,
Ayya Nadar Janaki Ammal College (Autonomous),
Sivakasi.

DECLARATION

This Internship report entitled “**Photographer Booking**”, has been carried out by me at the Post Graduate Department of Computer Applications, Ayya Nadar Janaki Ammal College (Autonomous), Sivakasi, affiliated to Madurai Kamaraj University, Madurai, in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications.

I further declare that this Internship report or any part thereof has not been submitted in this College or elsewhere for the award of any other degree or diploma.

Place: Sivakasi,
Date:

Signature of the Candidate
(R.PANDISELVAN)

ACKNOWLEDGEMENT

At the outset I thank **God** almighty for having brought, he will fulfilled power upon us in completing this Internship Programme.

I heartily express my sincere thanks to our College Management and our beloved Principal, **Dr. C. ASHOK, B.Sc., M.P.Ed., M.Phil., D.Y.MEd., Ph.D.**, for providing me this great opportunity and complete facility during this Internship Programme.

I heartily express my sincere thanks to **Dr. R. VENGATESHKUMAR, M.C.A., M.Phil., Ph.D.** Director, Post Graduate Department of Computer Applications for his valuable advice and guidance throughout this Internship Programme.

I heartily express my sincere thanks to **Mrs P. BARKAVI MCA., Associate Professor**, Post Graduate Department of Computer Applications for his/her excellence guidance, encouragement and personal commitments manifested throughout this Internship Programme.

I extend my sincere thanks to the Team leader of “**Abservetech Private Limited**” for their support throughout this Internship Programme.

I extend my sincere thanks to all our **department staff members** for their excellent support during this Internship Programme.

I also thank **my parents** for their encouragement and affection which helped us much to involve enthusiastically in our Internship Programme.

Place: Sivakasi,

R.PANDISELVAN

Date:

CHAPTER NO	CONTENTS	PAGE NO
1.	Synopsis	1
2.	System Requirements	2
	2.1 Hardware Requirements	2
	2.2 Software Requirements	2
3.	System Description	3
	3.1 Operating System	3
	3.2 Front End	4
	3.3 Back End	5
4.	System Design	8
	4.1 Data Flow Diagram	8
	4.2 Database Design	15
5.	System Implementation	22
	5.1 Modules Description	22
	5.2 Screenshots	25
6.	Conclusion	34
APPENDIX - A	BIBLIOGRAPHY	A1

1. SYNOPSIS

The system entitled "**Photographer Booking System**" is a web-based application designed to simplify the process of booking photographers for various events such as weddings, birthdays, engagements, corporate events, baby showers, and other special occasions. This platform provides an easy and convenient way for customers to search for photographers, view their profiles, explore their portfolios, and book their preferred photographer online. The application offers a simple and user-friendly interface that allows users to register, log in, select an event type, choose an available date, and confirm their booking within a few steps. Customers can also view their booking history and track the status of their bookings.

The system saves time by eliminating the need for phone calls and manual appointment scheduling. Photographers can create and manage their profiles by uploading their portfolios, updating their availability, and accepting or rejecting booking requests. This helps photographers organize their schedules efficiently and manage customer appointments without conflicts. The admin is responsible for managing users, photographers, bookings, and maintaining the overall system. The Photographer Booking System ensures accurate booking management by storing all information securely in a database. It reduces paperwork, minimizes booking errors, and improves communication between customers and photographers.

The application is developed using modern web technologies, providing fast performance, security, and reliability. Overall, the Photographer Booking System offers an efficient solution for both customers and photographers by making the booking process simple, quick, and hassle-free. It enhances customer satisfaction, improves business management, and provides a modern digital platform for photography services.

2. System Requirements

The system requirements include the hardware and software specifications needed to develop and run the application effectively. These requirements define the minimum and recommended hardware and software configurations necessary for the proper functioning of the Photographer Booking System. Meeting these requirements ensures that the application performs efficiently, provides a smooth user experience, and operates without technical issues. By satisfying the specified system requirements, users and administrators can install, access, and manage the application reliably while achieving optimal performance.

2.1 HARDWARE REQUIREMENTS

Processor	: AMD Ryzen 5 7520U with Radeon Graphics
RAM	: 16.00 GB
Hard disk	: 512 GB SSD

2.2 SOFTWARE REQUIREMENTS

Operating System	: Windows 10 / Windows 11
Frontend Technologies	: React.js
Backend Technologies	: Node.js, Express.js
Database	: MongoDB
Programming Language	: JavaScript
IDE / Editor	: Visual Studio Code
Web Browser	: Google Chrome

3. SYTEM DESCRIPTION

3.1 Operating System

Windows 11 is the latest major release of Microsoft's Windows NT operating system. It is the successor to Windows 10, which was released five years earlier. Windows 11 was officially announced on June 24, 2021, and became generally available on October 5, 2021. The operating system introduces a refreshed design and several new features aimed at improving user experience, performance, and security. It is available as a free upgrade for eligible Windows 10 devices via Windows Update. One of the most notable changes in Windows 11 is its redesigned user interface, featuring a centered Start menu and taskbar, rounded corners, and revamped system icons, giving it a modern and streamlined look. It also introduces features such as Snap Layouts and Snap Groups, which improve multitasking and window management. Virtual desktops have also been enhanced with better customization options.

In terms of hardware requirements, Windows 11 has stricter minimum system requirements than its predecessor. It requires a 64-bit processor with at least 1 GHz clock speed and two or more cores, 4 GB of RAM, and 64 GB of storage. Additionally, Windows 11 requires TPM (Trusted Platform Module) 2.0, UEFI Secure Boot support, and a DirectX 12-compatible graphics card for better security and performance. Because of these requirements, some older devices are not eligible for upgrading to Windows 11. Windows 11 places a stronger emphasis on integration with Microsoft's cloud services, including OneDrive, Microsoft 365, and Azure Active Directory, enabling improved productivity and collaboration. The Microsoft Store has been redesigned to support a wider range of applications, including Android apps through the Amazon Appstore, offering users more flexibility. Windows 11 also brings performance improvements such as better memory management, faster wake-from-sleep times, and improved battery efficiency for laptops. It also enhances gaming experiences through Xbox Game Pass integration, Auto HDR, and DirectStorage, which provide faster loading times and improved graphics performance.

As of recent updates, Windows 11 continues to grow in adoption as more devices meet its system requirements. However, Windows 10 will remain supported until October 14, 2025, giving users time to upgrade. Windows 11 is expected to evolve continuously with regular updates, bringing new features, security improvements, and performance enhancements.

3.2 Front End

3.2.1 React

React.js is a widely used JavaScript library for building user interfaces, especially single-page applications (SPAs). It is maintained by Meta (formerly Facebook) and a large developer community. React allows developers to build reusable UI components, making the development process more modular, efficient, and easier to maintain. One of the key features of React is its component-based architecture. In this approach, the entire user interface is divided into small, independent components. Each component manages its own logic and rendering, which makes the code reusable and easier to debug.

React follows a declarative programming style, meaning developers only describe what the UI should look like based on the current state. React automatically updates the interface whenever the state changes, reducing manual DOM manipulation. Another important feature of React is the Virtual DOM. Instead of directly modifying the real DOM, React creates a virtual representation of the UI. When changes occur, React compares the new Virtual DOM with the previous version and updates only the changed elements in the real DOM. This process significantly improves performance and speed. React also supports hooks such as `useState` and `useEffect`, which allow functional components to manage state and lifecycle events efficiently. This has reduced the need for class-based components and made development simpler.

In modern web development, React is often combined with libraries like React Router for navigation and Redux or Context API for state management. It is also widely used with backend technologies such as Node.js to create full-stack applications. Due to its flexibility, performance, and strong community support, React is used in many popular applications like Facebook, Instagram, Netflix, and Airbnb.

3.2.3 TAILWIND CSS

Tailwind CSS is a utility-first CSS framework used to design modern and responsive user interfaces quickly. Unlike traditional CSS frameworks that provide pre-designed components, Tailwind offers low-level utility classes that can be directly applied in HTML or JSX. This approach allows developers to build custom designs without writing large CSS files. Each utility class in Tailwind represents a specific style such as margin, padding, color, font size, or layout. This makes the design process faster and more consistent.

Tailwind CSS supports mobile-first responsive design, meaning developers can easily define how elements should behave on different screen sizes using responsive prefixes. Another major advantage of Tailwind is its customization system. Developers can configure colors,

spacing, fonts, and breakpoints according to project requirements. This helps in maintaining a consistent design system across the entire application.

Tailwind also integrates smoothly with modern frontend frameworks like React, Angular, and Vue. It works well with build tools like PostCSS and can remove unused styles in production using PurgeCSS, resulting in optimized and lightweight CSS files. Overall, Tailwind CSS improves productivity and reduces the need for writing repetitive CSS code, making it ideal for modern web applications.

3.3 Back End

3.3.1 EXPRESS JS

Express.js is a fast, minimal, and flexible web application framework for Node.js, designed to simplify the process of building robust and scalable server-side applications. It was originally released in 2010 and quickly became one of the most popular frameworks in the Node.js ecosystem due to its simplicity and powerful features for building web and mobile applications.

Express is lightweight, flexible, and unopinionated, meaning developers have complete freedom to structure their applications according to their requirements without being restricted to strict rules or predefined architecture. This makes it suitable for both small-scale and large-scale enterprise applications. Express provides a simple and efficient interface for handling HTTP requests and responses. It supports routing, middleware, and template engines, which are used for rendering dynamic web content. Routing helps manage different URLs and define how an application responds to client requests, while middleware functions are used to process requests before sending responses.

The framework is widely used for building RESTful APIs, single-page applications (SPAs), and full-stack web applications. It allows easy integration with databases, file systems, and external APIs, making it suitable for applications that require authentication, real-time communication, and data processing. Middleware functions in Express provide powerful features such as logging, error handling, authentication, and session management. These functions help in extending the functionality of the application without modifying the core logic. Due to its simplicity and efficiency, Express has become a core component of the MEAN and MERN stacks (MongoDB, Express, Angular/React, Node.js). It is widely adopted by developers for building scalable and high-performance web applications.

Express is open-source and supported by a large developer community. It is regularly updated to improve security, performance, and usability. It is considered one of the most trusted

backend frameworks for Node.js developers because of its fast learning curve and ability to handle high-traffic applications efficiently.

HISTORY

Express was created by TJ Holowaychuk in 2010 as part of his effort to simplify web application development using Node.js. It was initially designed as a minimal framework for building HTTP servers. Over time, it evolved into a powerful web framework used in many modern applications. Due to its simplicity, flexibility, and strong ecosystem of plugins and middleware, Express quickly gained popularity among developers. Today, it remains one of the most widely used frameworks in server-side development and continues to be actively maintained by the open-source community.

3.3.2 NODE JS

Node.js is a powerful, open-source, cross-platform runtime environment that allows developers to run JavaScript on the server side. It was originally developed by Ryan Dahl in 2009. Node.js uses the V8 JavaScript engine, which is the same engine used in Google Chrome, to execute JavaScript code outside the browser. This makes it possible for developers to use JavaScript for both client-side and server-side programming, providing a unified and efficient development experience.

One of the greatest strengths of Node.js is its large ecosystem of libraries and packages available through npm (Node Package Manager), which is the largest package manager in the world. This allows developers to quickly add pre-built functionality to their applications, such as authentication, database connectivity, file handling, and API integration, thereby reducing development time.

Node.js is commonly used for building web servers, APIs, microservices, real-time applications such as chat applications, gaming servers, and collaboration tools, as well as command-line tools. Its ability to handle a large number of concurrent connections efficiently makes it highly suitable for high-traffic and scalable applications.

Because Node.js uses JavaScript, it is preferred by full-stack developers who can work on both frontend and backend using the same language. It also integrates well with modern frontend frameworks such as React, Angular, and Vue.js, making it a popular choice for building dynamic and responsive web applications.

HISTORY OF NODE.JS

Node.js was created in 2009 by Ryan Dahl to overcome the limitations of traditional server-side technologies that relied on blocking I/O operations. His goal was to build a runtime

environment capable of handling multiple concurrent connections using a non-blocking, event-driven architecture.

Over time, Node.js gained strong community support due to its performance and scalability. In 2011, npm (Node Package Manager) was introduced, which significantly expanded the Node.js ecosystem by allowing developers to share and reuse code easily.

Today, Node.js is one of the most widely used backend technologies in modern web development and continues to evolve with regular updates and strong open-source community support.

3.3.3 MONGODB

MongoDB is a widely used, open-source, NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON). It is designed to handle large volumes of unstructured or semi-structured data efficiently, making it suitable for modern data-driven applications. Unlike traditional relational databases, MongoDB uses a flexible schema instead of fixed tables. This allows developers to store and modify data easily without strict structure limitations. MongoDB stores data in the form of documents inside collections.

Each document is a JSON-like structure containing key-value pairs, and collections hold multiple documents that may have different fields. One of the main features of MongoDB is horizontal scaling using sharding, where data is distributed across multiple servers for better performance and reliability. It also provides replication for data backup and high availability.

MongoDB supports powerful querying features such as filtering, indexing, full-text search, and aggregation. The aggregation framework allows developers to perform complex data analysis and transformations. MongoDB is widely used with Node.js and Express.js in modern web development. It is a key component of MERN (MongoDB, Express, React, Node.js) and MEAN (MongoDB, Express, Angular, Node.js) stacks.

MongoDB was created in 2007 by Dwight Merriman, Eliot Horowitz, and Kevin Ryan and released in 2009. It has become one of the most popular NoSQL databases due to its flexibility, scalability, and ease of use.

4. SYSTEM DESIGN

4.1 DATA FLOW DIAGRAM:

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. It is used to model how data moves from one process to another within a system and how it is processed at different stages. DFDs are often used in the early stages of system design to provide a clear overview of the system structure. They help developers and analysts understand how data enters the system, how it is processed, and how the output is generated. A DFD can later be expanded into more detailed system designs.

DFDs are widely used because they are simple, easy to understand, and effective for visualizing data movement within a system.

CONTEXT DIAGRAM:

A Context Diagram is a high-level representation of a system, also known as a **Level 0 Data Flow Diagram (DFD)**. It provides an overall view of the entire system and its interaction with external entities. In a Context Diagram, the whole system is represented as a single process called **“Process 0”**, which represents the complete functionality of the system.

This process shows how the system interacts with external entities such as users, customers, or other external systems. The main purpose of a Context Diagram is to define the system boundary and clearly show what lies inside and outside the system. It helps in understanding the input and output flow between the system and external sources.

Unlike detailed DFDs, a Context Diagram does not show internal processes or data storage. It only focuses on the highest level view of the system.

DFD LAYERS AND LEVELS:

Data Flow Diagrams (DFDs) can be developed in multiple layers of detail. A single process in a high-level diagram can be expanded into a more detailed DFD in lower levels. This layered approach helps in understanding the system step by step, starting from a simple overview and moving toward detailed process design.

The first step in creating a DFD is to draw the **Context Diagram**, which represents the entire system as a single process. After that, the system is broken down into multiple levels of DFDs to show more detailed internal processes and data flows.

DFD LEVELS:**Level 0 DFD (Context Diagram):**

The Level 0 DFD, also known as the Context Diagram, shows the entire system as a single process. It represents how the system interacts with external entities such as users or other systems. This level provides a simple overview of the system boundary and its external interactions.

Level 1 DFD:

The Level 1 DFD breaks down the main system process into multiple sub-processes. Each process represents a major function of the system. It shows how data flows between different processes, external entities, and data stores.

Level 2 DFD:

The Level 2 DFD provides a more detailed breakdown of Level 1 processes. Each sub-process is further divided into smaller processes to show detailed data flow and system behavior.

DATA FLOW DIAGRAM (DFD) SYMBOLS:

Data Flow Diagrams use standard symbols to represent the flow of data and the components of a system. These symbols help in visually representing how data moves, how it is processed, and where it is stored in a system. Each symbol has a specific meaning and function in the diagram.

1. External Entity (Source or Destination):

External entities represent sources or destinations of data that exist outside the system. They provide input to the system or receive output from it.

- **Symbol:** Rectangle
- **Representation:** User, Customer, Admin, External System
- **Function:** Sends input to the system or receives output from the system

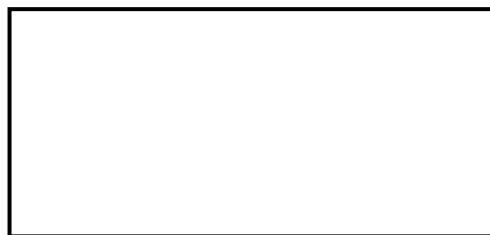


Figure: 4.1.1 Entity

2. Process

process represents an activity or operation that transforms input data into output data.

- **Symbol:** Circle or Oval
- **Representation:** Actions like “Login”, “Register”, “Process Order”
- **Function:** Converts input data into meaningful output

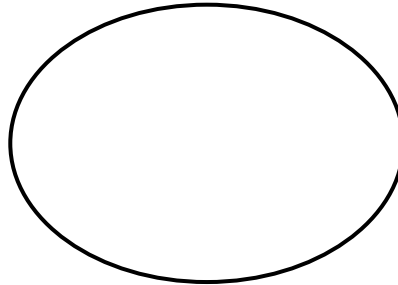


Figure: 4.1.2 Process

3. Data Flow

Data flow shows the movement of data between processes, external entities, and data stores.

- **Symbol:** Arrow (→)
- **Representation:** Direction of data movement
- **Function:** Indicates how data flows in the system



Figure: 4.1.3 Data Flow

4. Data Store

A data store represents a storage location where data is kept for future use.

- **Symbol:** Open-ended rectangle or two parallel lines
- **Representation:** Database, File, Table
- **Function:** Stores data that can be accessed later by processes



Figure: 4.1.4 Data Store

Level 0

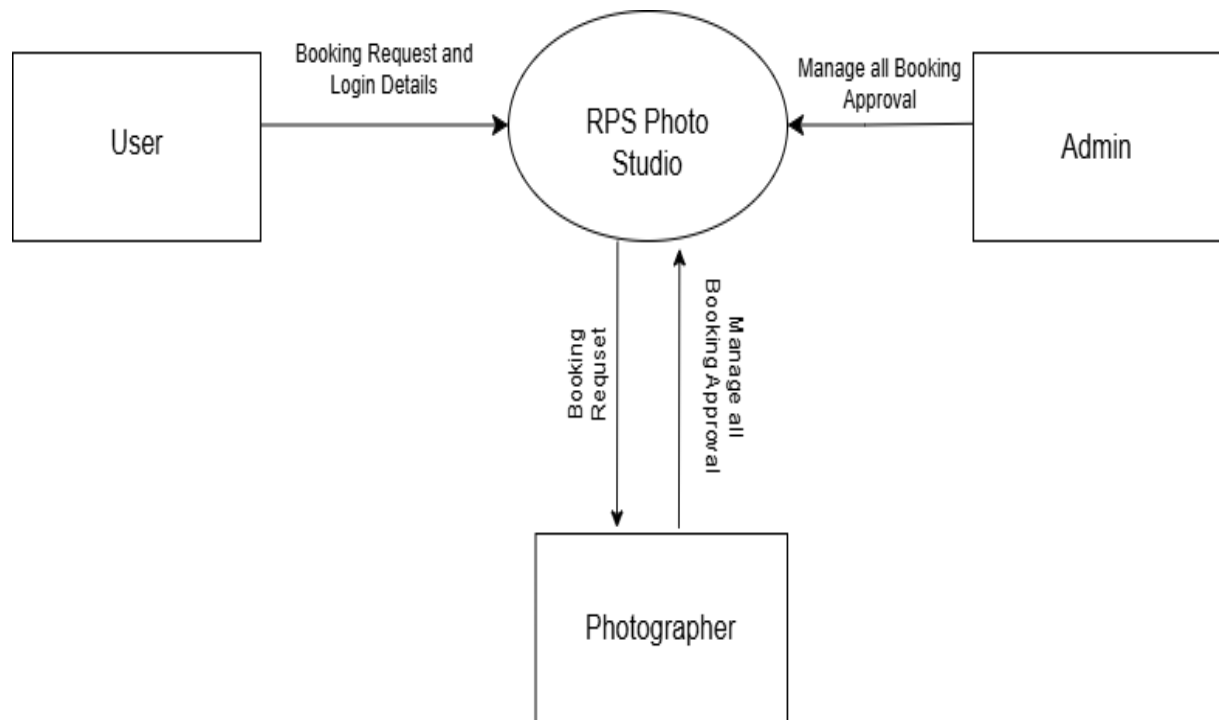


Figure: 4.1.5 Context level Diagram

Level 1.1

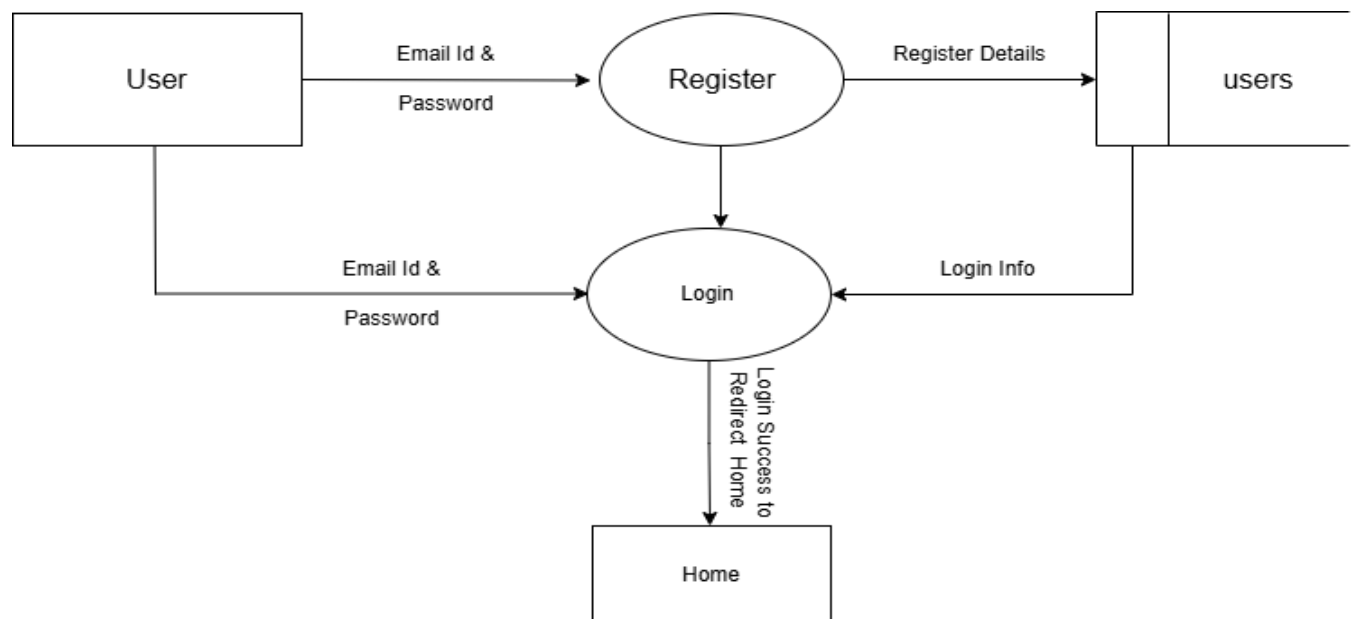


Figure: 4.1.6 User Login

Level 1.2

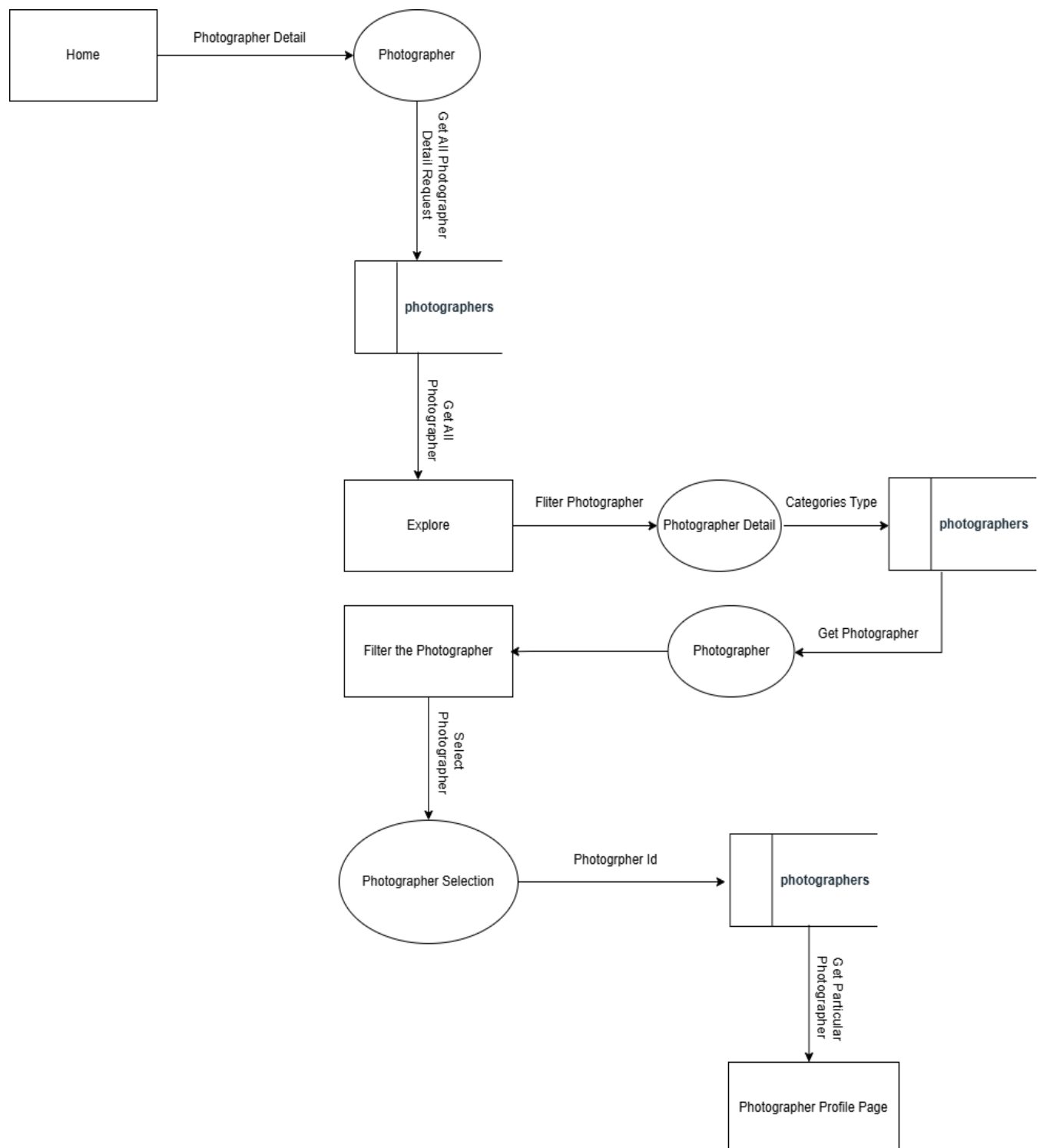


Figure: 4.1.7 Photographer Selection

Level 1.3

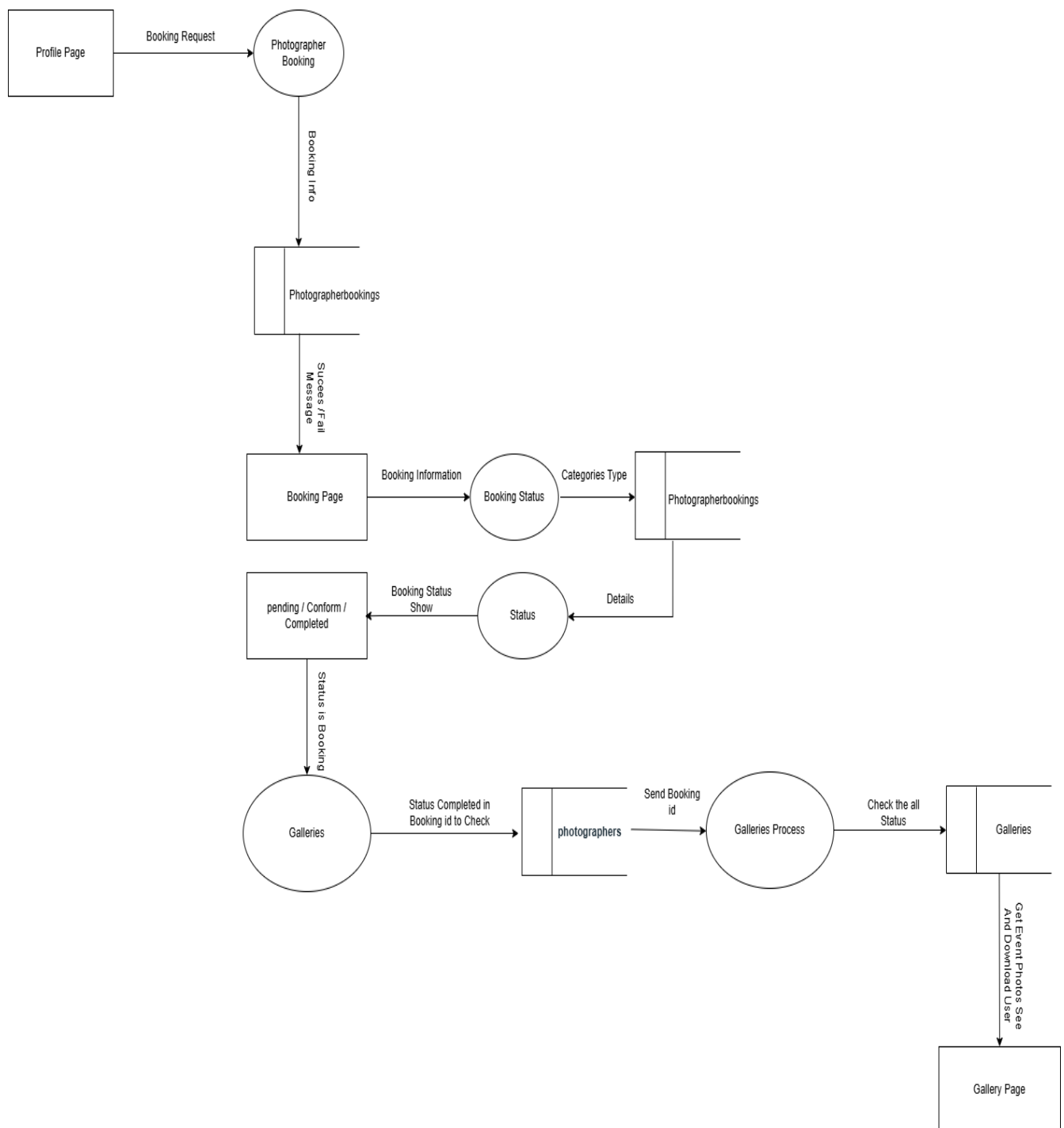


Figure: 4.1.7 Booking Status and User Event Gallery

Level 2.1

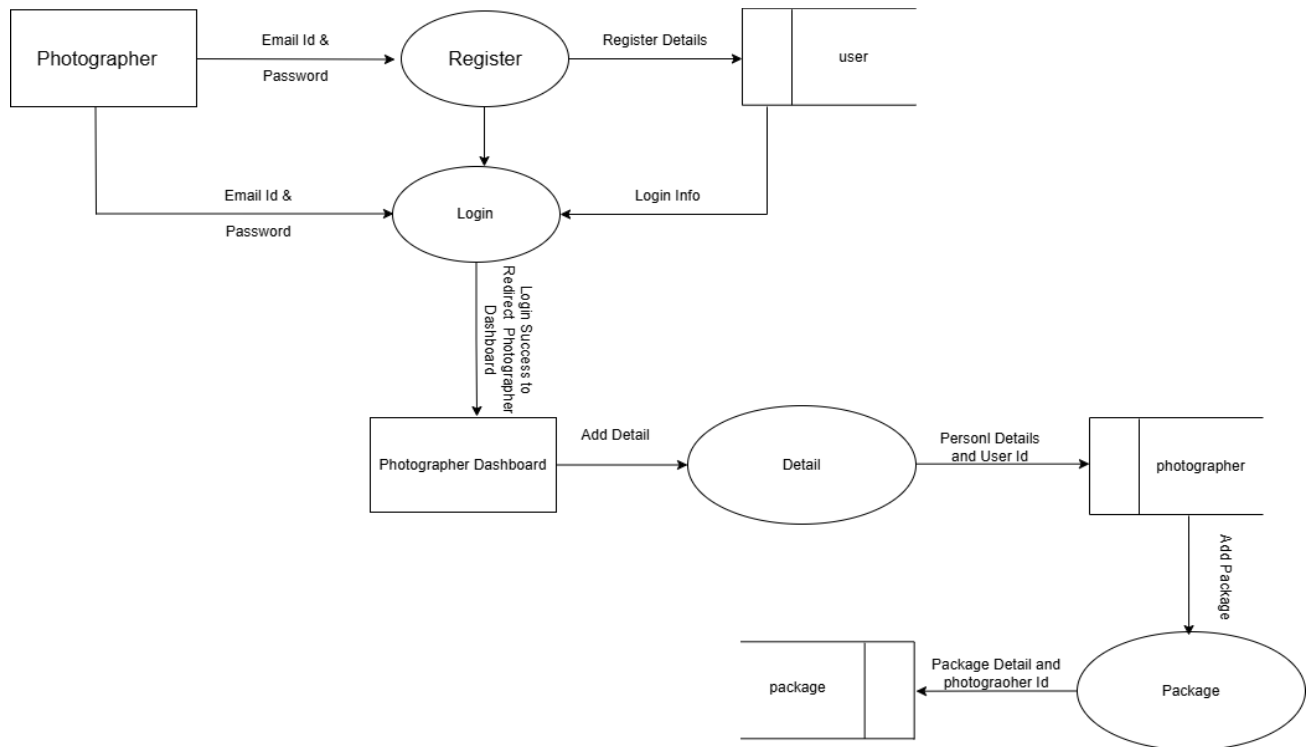


Figure: 4.1.8 Photographer Login / Register and Photographer add Package

Level 2.2

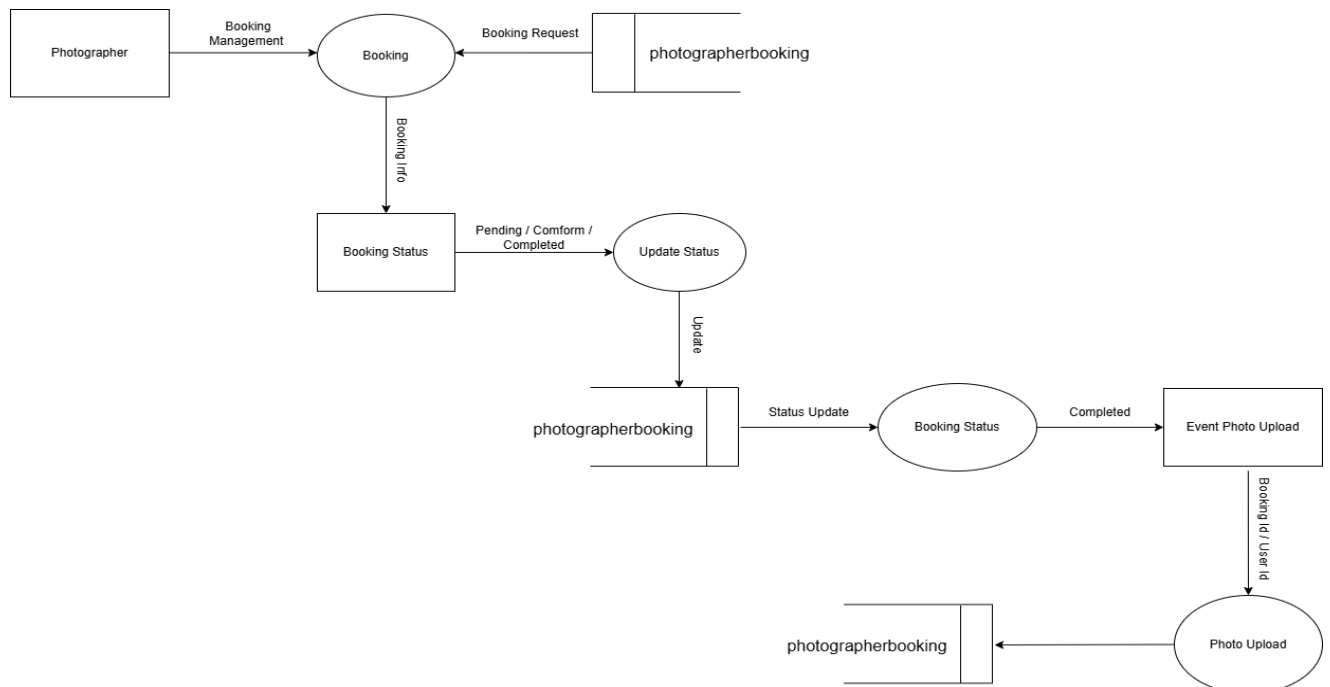


Figure: 4.1.9 Photographer Booking Status Update and Event Photo Upload

4.2 DATABASE DESIGN:

A database is one of the most important components of the **Photographer Booking System**. It is a collection of related data that is organized and stored efficiently to provide fast, secure, and reliable access to information. The main objective of the database is to minimize data redundancy, maintain data consistency, and allow multiple users to access the system simultaneously.

The Photographer Booking System uses **MongoDB**, a NoSQL database, to store and manage all application data. MongoDB stores information in the form of collections and documents, providing flexibility and high performance. Since MongoDB is schema-less, it allows the application to store different types of data without requiring changes to the database structure. This makes it suitable for modern web applications where data requirements may change over time. Database design is an important phase of application development because it ensures efficient data storage, retrieval, and management. A well-designed database improves application performance, maintains data integrity, and supports future expansion of the system.

An **entity** represents a real-world object or concept within the application. In the Photographer Booking System, the main entities include **User**, **Photographer**, **Booking**, **Payment**, **Category**, **Portfolio**, **Review**, and **Admin**. Each entity contains several **attributes** that describe its characteristics. For example, the Photographer entity includes Photographer ID, Name, Email, Phone Number, Experience, Location, Category, Price, and Portfolio Images. The **User** entity stores customer information such as User ID, Name, Email, Phone Number, Password, and Address. The **Booking** entity stores booking details such as Booking ID, Booking Date, Event Type, Event Location, Booking Status, User ID, and Photographer ID. The **Payment** entity stores payment details including Payment ID, Amount, Payment Method, Transaction ID, Payment Date, and Payment Status.

The database maintains relationships between different collections to ensure proper communication among system modules. A registered user can search photographers, view portfolios, check availability, book a photographer, make online payments, and submit reviews after the event. The administrator can manage photographers, users, bookings, categories, payments, and customer feedback through the admin dashboard. The database is designed to reduce data redundancy and maintain consistency. Although MongoDB does not require traditional normalization, the collections are organized logically to avoid unnecessary duplication of information. Proper indexing is implemented on frequently searched fields such as photographer name, category, location, and booking date to improve query performance and reduce response time.

Security is another important aspect of database design. User passwords are stored securely using encryption techniques. Authentication and authorization mechanisms ensure that only authorized users and administrators can access sensitive information. Data validation is performed before storing information in the database to maintain accuracy and prevent invalid data entries. The database is also designed for scalability so that new features such as online chat, photographer availability calendars, discount coupons, notification systems, event galleries, and customer feedback analysis can be added without major modifications to the existing database structure. MongoDB's flexible document model makes it easy to accommodate future enhancements.

Overall, the database design provides a secure, scalable, and efficient foundation for the Photographer Booking System. It supports fast data retrieval, reliable booking management, secure payment processing, and effective administration while ensuring data consistency and high system performance.

Table: 4.2.1 user

Field Name	Data Type	Constraints	Description
UserName	String	Required	Name of the user
password	String	Required, Minlength: 8	User login password
role	String	Enum (User, Admin, Photographer), Default: User	User role
MobileNumber	String	Required	User mobile number
address	String	Default: empty	User address
city	String	Default: empty	City
zipCode	String	Default: empty	Postal code
profileImage	Object	public_id, url	Profile image
profileMedia	Array	public_id, url, mimeType, size	User media files
EmailId	String	Required, Unique, Valid Email	User email

resetPasswordToken	String	Optional	Password reset token
resetPasswordExpire	Date	Optional	Token expiry time
emailVerifyCode	String	Optional	Email verification code
emailVerifyExpire	Date	Optional	Verification expiry
isEmailVerified	Boolean	Default: false	Email verification status

Table: 4.2.2 photographer

Field Name	Data Type	Constraints	Description
user	ObjectId	Ref: user	Linked user account
name	String	Required	Photographer name
email	String	Optional	Email
contactNumber	String	Optional	Phone number
speciality	String	Default: General	Photography type
experience	Number	Default: 0	Years of experience
location	String	Optional	Working location
price	Number	Optional	Service price
bio	String	Default: General	Description
portfolio	Array	public_id, url	Work samples
BackgroundImage	Array	public_id, url	Background images
ProfileImage	Array	public_id, url	Profile images
packages	Array	Ref: Package	Service packages

Table: 4.2.3 Package

Field Name	Data Type	Constraints	Description
photographerId	ObjectId	Ref: photographers, Required	Photographer
packageName	String	Required	Package name
tier	String	Enum (Premium, Gold, Silver)	Package level
description	String	Required	Package details
price	Number	Required	Cost
features	Array	String list	Included features
duration	String	Required	Time duration
status	String	Enum (Active, Inactive)	Availability

Table: 4.2.4 review

Field Name	Data Type	Constraints	Description
photographerId	ObjectId	Ref: photographers, Required	Photographer
userId	ObjectId	Ref: user, Required	Reviewer
rating	Number	Min:1, Max:5	Rating score
comment	String	Required	Review comment
createdAt	Date	Auto	Created time
updatedAt	Date	Auto	Updated time

Table: 4.2.5 gallery

Field Name	Data Type	Constraints	Description
UserId	ObjectId	Ref: user, Required	User
photographerId	ObjectId	Ref: photographers, Required	Photographer
bookingId	ObjectId	Ref: PhotoGrapherBooking	Booking reference
eventName	String	Required	Event name
images	Array	url, public_id	Event photos
isPublic	Boolean	Default: true	Visibility

Table: 4.2.6 PhotoGrapherBooking

Column Name	Data Type	Constraints	Description
_id	ObjectId	Primary Key, Auto-generated	Unique booking ID (MongoDB default)
fullName	String	Required	Customer full name
email	String	Required	Customer email address
phoneNumber	String	Required	Primary contact number
alternateNumber	String	Optional	Alternative contact number
sessionType	String	Required	Type of photography session
preferredDate	Date	Required	Booking preferred date
preferredTime	String	Required	Preferred time slot
duration	String	Required	Session duration
eventLocation	String	Required	Event location address
city	String	Required	City of event
zipCode	String	Required	Postal/ZIP code
specialRequests	String	Optional	Extra customer requirements
photographerId	ObjectId	Required, Ref: photographers	Selected photographer ID

requestedPhotographerId	ObjectId	Optional, Ref: photographers	Alternative requested photographer
bookingMode	String	Enum: standard, extra, refer; Default: standard	Type of booking
availabilityOverride	Boolean	Default: false	Extra availability request
extraAvailabilityFee	Number	Default: 0	Additional availability charge
referralFee	Number	Default: 0	Referral-based fee
UserId	ObjectId	Required, Ref: user	User who booked
packageName	String	Required	Selected package name
packageTier	String	Enum: Premium, Gold, Silver	Package category
packagePrice	Number	Required	Base package cost
travelCost	Number	Default: 0	Travel charges
taxes	Number	Default: 0	GST / tax amount
totalAmount	Number	Required	Final payable amount
paymentMode	String	Enum: razorpay_test, manual, pending; Default: pending	Payment method
paymentStatus	String	Enum: Pending, Paid, Failed; Default: Pending	Payment status
razorpayPaymentId	String	Optional	Razorpay transaction ID
razorpayOrderId	String	Optional	Razorpay order ID
razorpaySignature	String	Optional	Razorpay verification signature

status	String	Enum: Pending, Confirmed, Completed, Cancelled; Default: Pending	Booking status
createdAt	Date	Auto-generated	Record creation timestamp
updatedAt	Date	Auto-updated	Last update timestamp

5. SYSTEM IMPLEMENTATION

5.1 MODULE DESCRIPTION

The project is entitled “Photographer Booking” developed using React JS as the frontend and Node.js, Express.js, and MongoDB as the backend technologies. The project has the following modules:

- Administrator Module
- User Module
- Photographer Module

Administrator Module:

In this module, the administrator can manage the entire web application and monitor user details, photographer details, booking details, and payment information. The administrator module contains the following sub-modules such as:

- Login Module
- Booking Info
- User Info
- Photographer Info

Login Module:

This module allows the admin to enter their username and password to log in to the admin dashboard.

Photographer Info:

In this module, the admin can view photographer details and also add, delete, and update photographer profiles, including experience, specialization, package details, and availability.

Booking Info:

This module allows the administrator to access booking information, including event type, booking date, selected photographer, and payment status.

User Info:

This module contains the basic information about users who are already registered in the web application. The admin can view how many users are registered. If a new user registers or user details are updated, the system automatically updates the data.

User Module:

In this module, users can browse photographer profiles, check their portfolio, and book photographers for different events such as weddings, birthdays, and other functions. Users can register in the web application by entering their username, email, and password. They can select packages, confirm bookings, and make payments securely. The user module contains sub-modules such as:

- Login Module
- Register Module
- Photographer Module
- Booking Module
- Payment Module

Login Module:

The Login Module allows users to log in by entering their username and password. It works with authentication to verify whether the user already exists or not.

Register Module:

The Register Module allows users to create an account by entering their name, email, and password. It ensures password security by hashing. This module helps with authentication and access control and includes input validation to prevent invalid or duplicate registrations.

Photographer Module:

In this module, users can search for photographers based on category, location, and event type. Users can view photographer details such as experience, sample works, ratings, and package prices before booking.

Booking Module:

In the Booking Module, users can select their preferred date, event type, and package details. They can confirm the booking and view booking history. The system stores all booking details for future reference.

Payment Module:

The Payment Module ensures secure and smooth transactions for customers. It supports multiple payment methods such as UPI, debit/credit cards, net banking, and wallets through **Razorpay** integration. This module uses encryption to protect payment details and ensures secure payment processing.

Photographer Module

In this module, photographers can register and manage their profiles, upload their portfolio, update their availability, and handle booking requests from users. This module helps photographers maintain their professional details and communicate with clients efficiently. The photographer module contains the following sub-modules such as:

- Registration Module
- Login Module
- Manage Profile
- Upload Portfolio
- Manage Availability
- View Booking Requests
- Accept / Reject Booking
- Earnings Module
- Gallery Upload Module

Registration Module:

The Registration Module allows photographers to create an account by entering their basic details such as name, email, phone number, experience, and specialization. This module stores the photographer's information securely in the database.

Login Module:

The Login Module allows photographers to log in using their username and password. It verifies authentication and provides secure access to their dashboard.

Manage Profile:

In this module, photographers can update their personal details, specialization, event categories, package prices, and contact information. This helps keep their profile updated for users.

Upload Portfolio:

The Upload Portfolio Module allows photographers to upload sample photos of their previous work. Users can view these portfolio images before booking, helping them choose the right photographer.

Manage Availability:

In this module, photographers can update their available dates and timings for bookings. This prevents booking conflicts and helps users check availability easily.

View Booking Requests:

This module allows photographers to view all booking requests sent by users. It contains details such as event type, booking date, location, and selected package.

Accept / Reject Booking:

In this module, photographers can accept or reject booking requests based on their availability. Once the decision is made, the booking status is updated automatically.

Earnings Module:

The Earnings Module allows photographers to view their completed bookings and total earnings. It helps them track their income and payment history.

Gallery Upload Module:

In this module, users can upload their event photos after the completion of the function. These photos will be stored in the system and linked to that particular user's account. The user can view, download, and manage their personal event gallery anytime. This module helps users keep their memories safe and easily accessible in one place. Only the respective user can access their uploaded event photos, ensuring privacy and security.

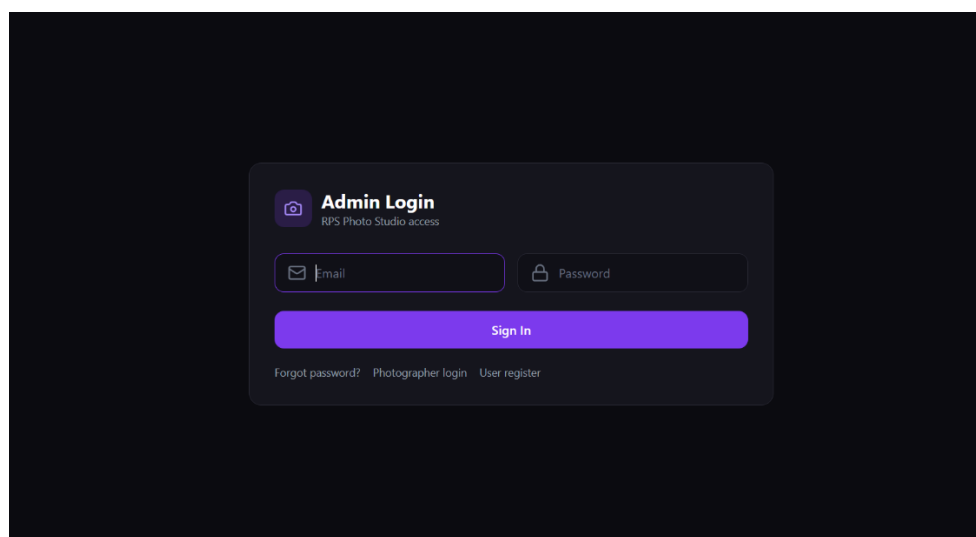
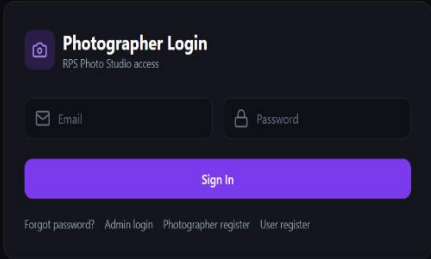
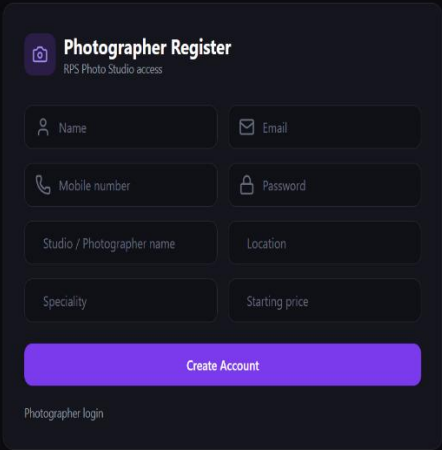
5.2 SCREEN SHOTS

Figure: 5.2.1 Admin Login



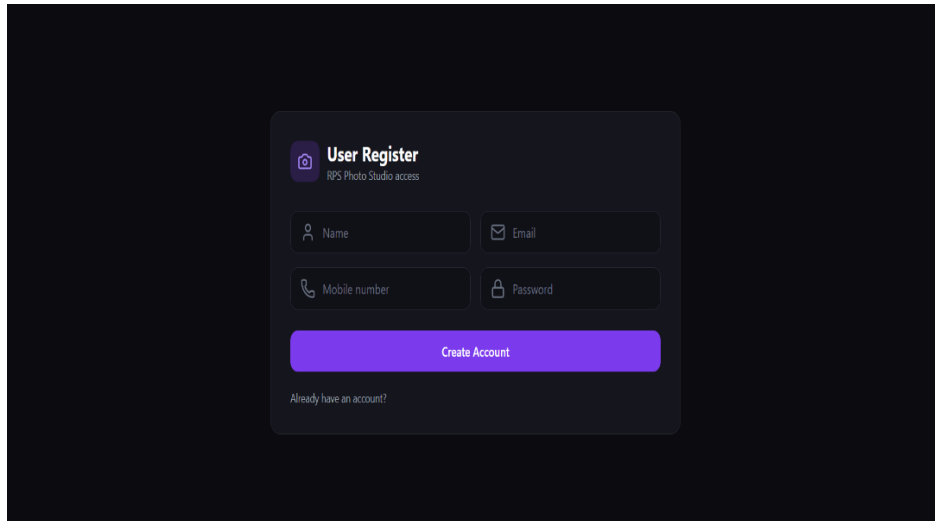
The image shows a 'Photographer Login' form on a dark background. The form has a title 'Photographer Login' with a camera icon and subtitle 'RPS Photo Studio access'. It contains two input fields: 'Email' with an envelope icon and 'Password' with a lock icon. Below these is a large blue 'Sign In' button. At the bottom, there are links: 'Forgot password?', 'Admin login', 'Photographer register', and 'User register'.

Figure: 5.2.2 Photographer Login



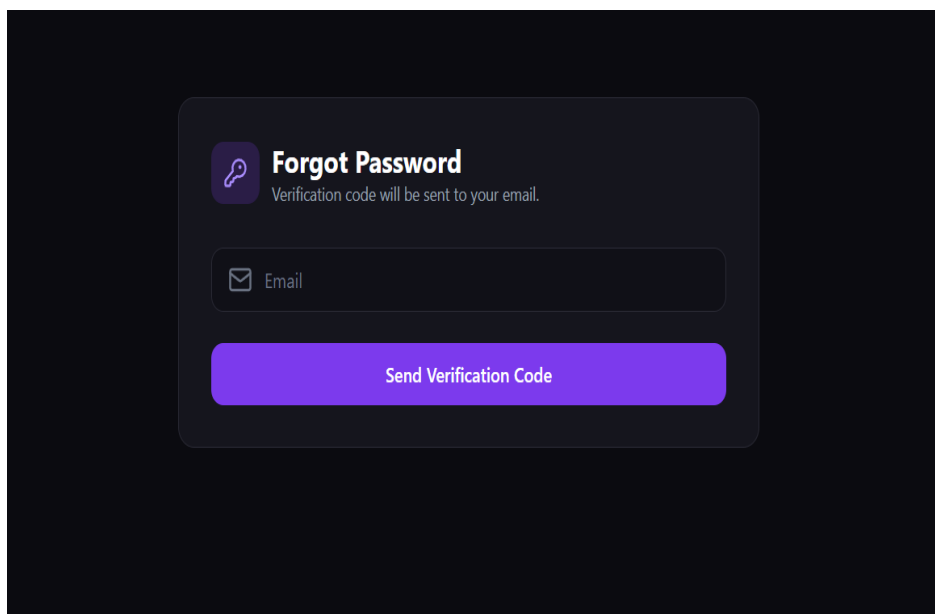
The image shows a 'Photographer Register' form on a dark background. The form has a title 'Photographer Register' with a camera icon and subtitle 'RPS Photo Studio access'. It contains six input fields arranged in three rows: 'Name' (with a person icon), 'Email' (with an envelope icon), 'Mobile number' (with a phone icon), 'Password' (with a lock icon), 'Studio / Photographer name', and 'Location'. Below these is a large blue 'Create Account' button. At the bottom, there is a link: 'Photographer login'.

Figure: 5.2.3 Photographer Register



The image shows a 'User Register' form on a dark background. The form is titled 'User Register' with a subtitle 'RPS Photo Studio access'. It contains four input fields: 'Name', 'Email', 'Mobile number', and 'Password'. Each field has a corresponding icon (person, envelope, phone, and lock respectively). Below the fields is a large orange button labeled 'Create Account'. At the bottom, there is a link that says 'Already have an account?'.

Figure: 5.2.4 User Register



The image shows a 'Forgot Password' form on a dark background. The form is titled 'Forgot Password' with a subtitle 'Verification code will be sent to your email.' It contains one input field labeled 'Email' with an envelope icon. Below the field is a large orange button labeled 'Send Verification Code'.

Figure: 5.2.5 Forgot Password Page

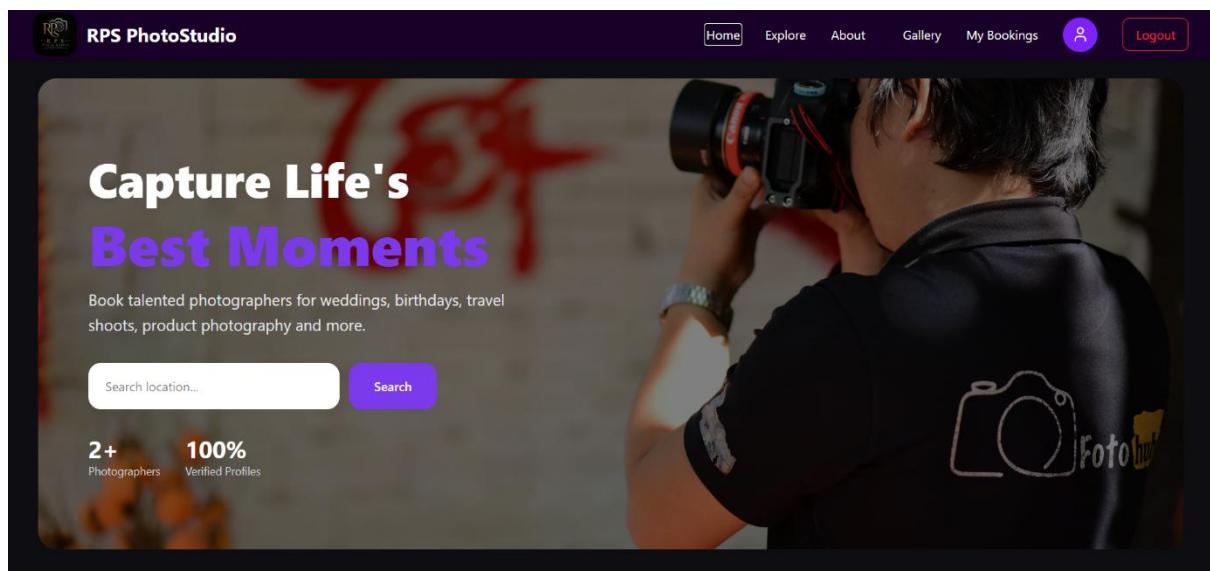


Figure: 5.2.6 User Home Page

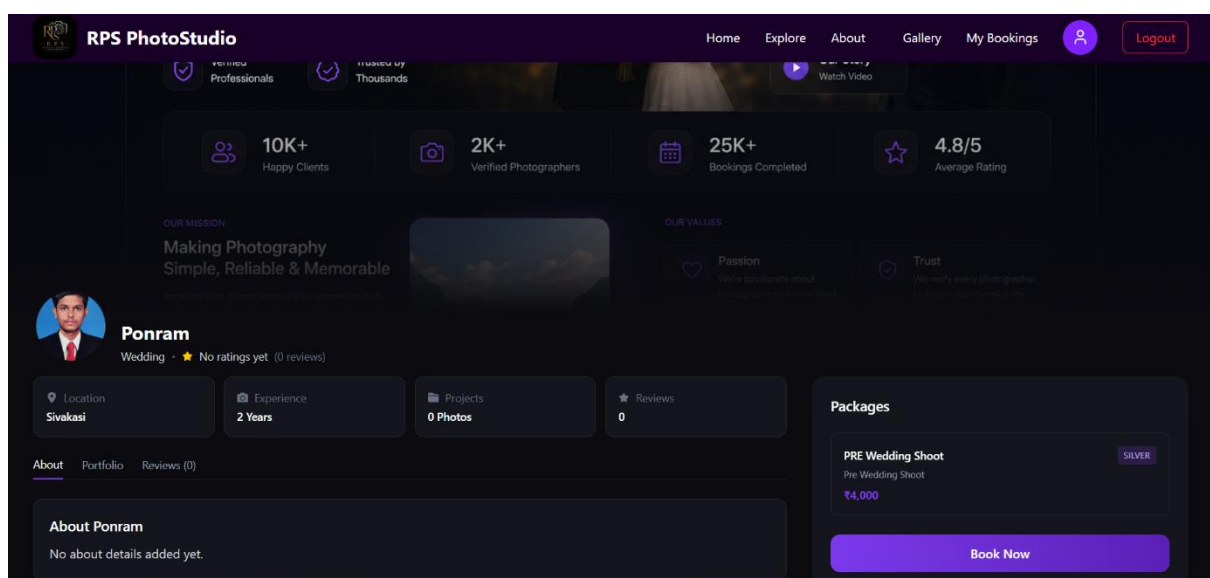


Figure: 5.2.7 Photographer Profile Page

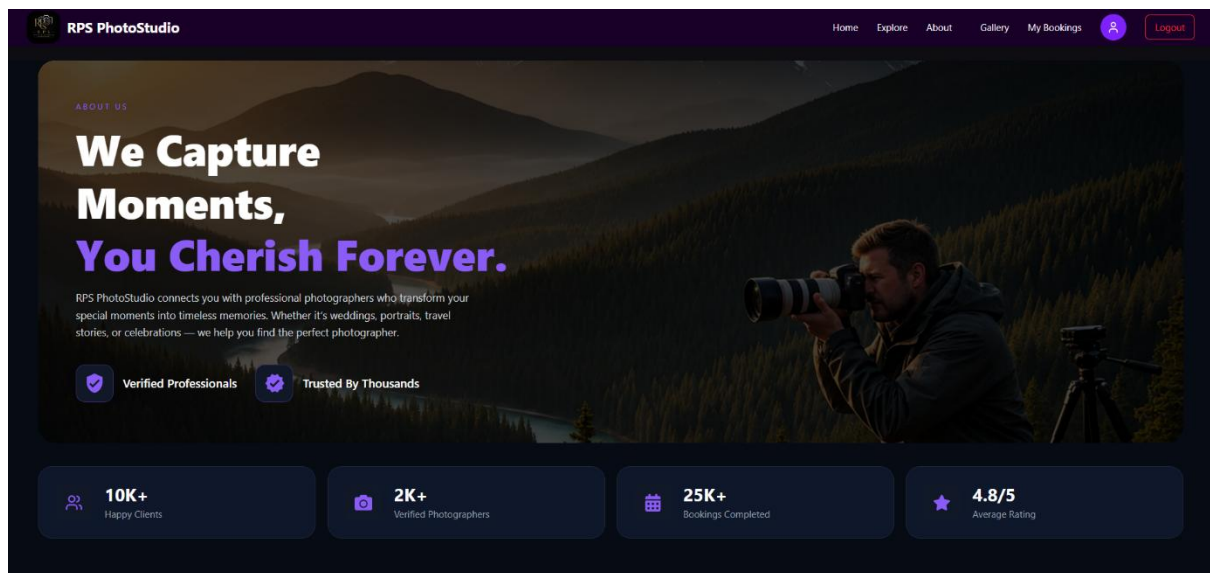


Figure: 5.2.8 About Page

Booking Details

Provide your information and session details

Personal Information

Session Information

Location Details

Booking Summary

Review your booking details



Ponram
No Rating
Sivakasi

Selected Package

PRE Wedding Shoot **Rs.4000**
SILVER
Pre Wedding Shoot

Price Summary

Package Price	Rs.4000
Travel Cost	Rs.1000
Taxes & Fees	Rs.0
Total Amount	Rs.5000

Figure: 5.2.9 Photographer Booking Page

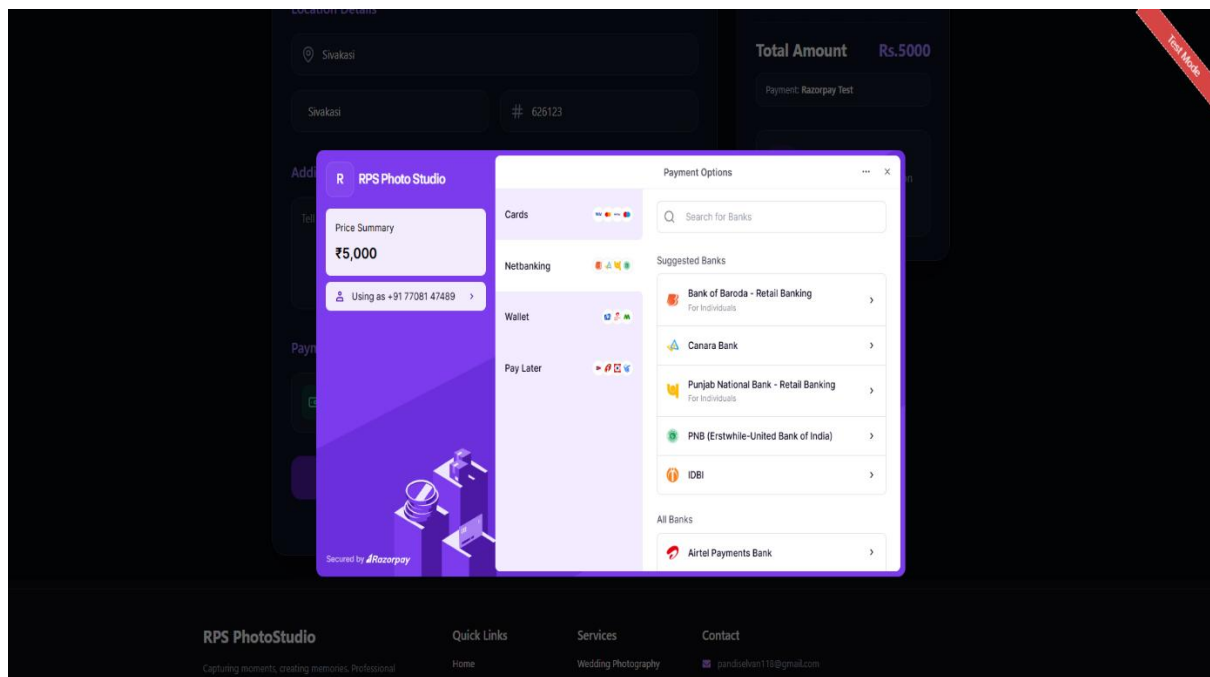


Figure: 5.2.10 Payment Page

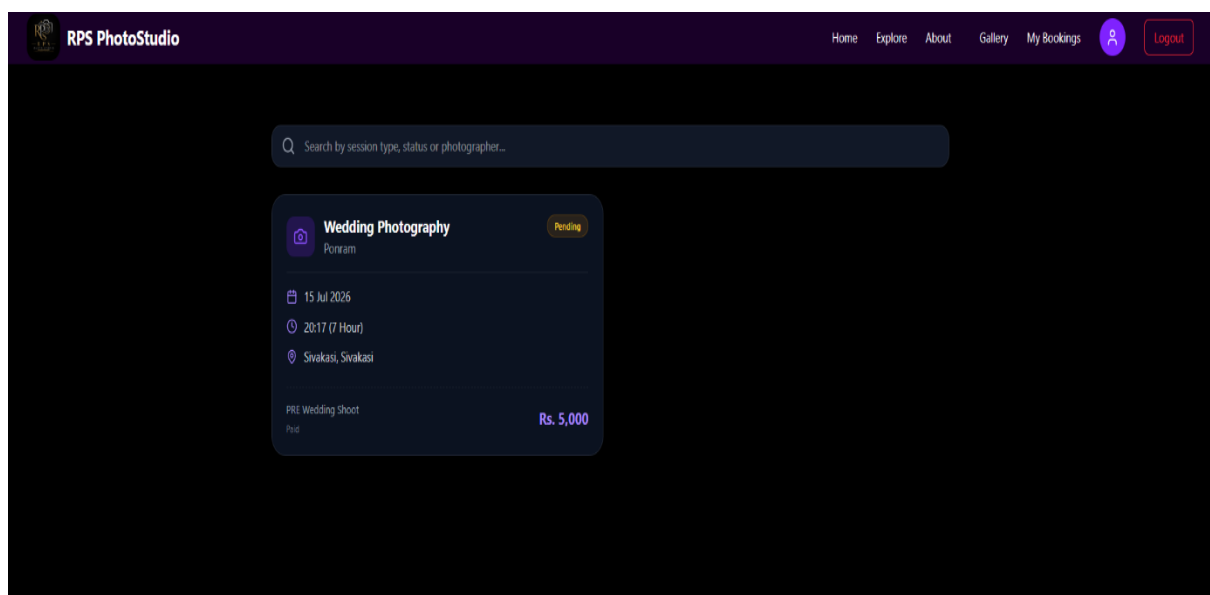


Figure: 5.2.11 Booking Detail Page

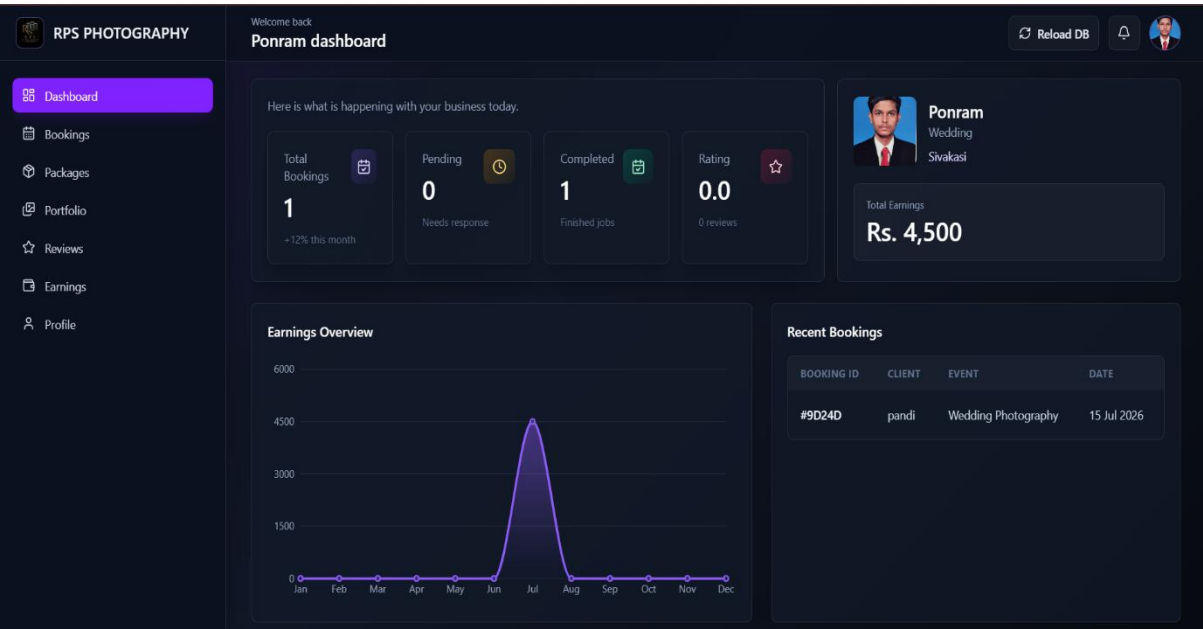


Figure: 5.2.12 Photographer Dashboard

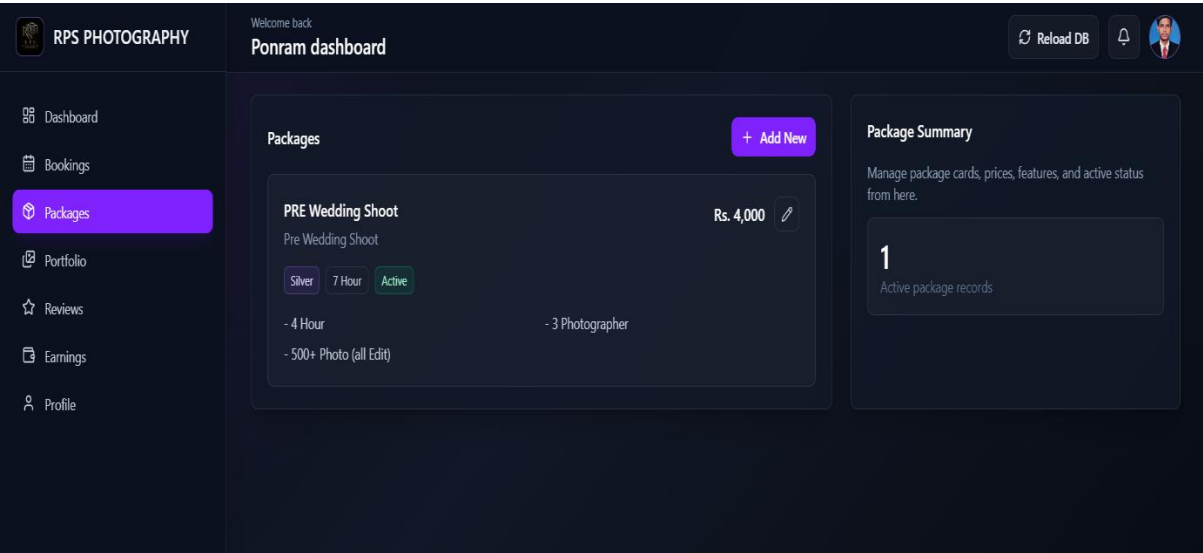


Figure: 5.2.13 Add Packages

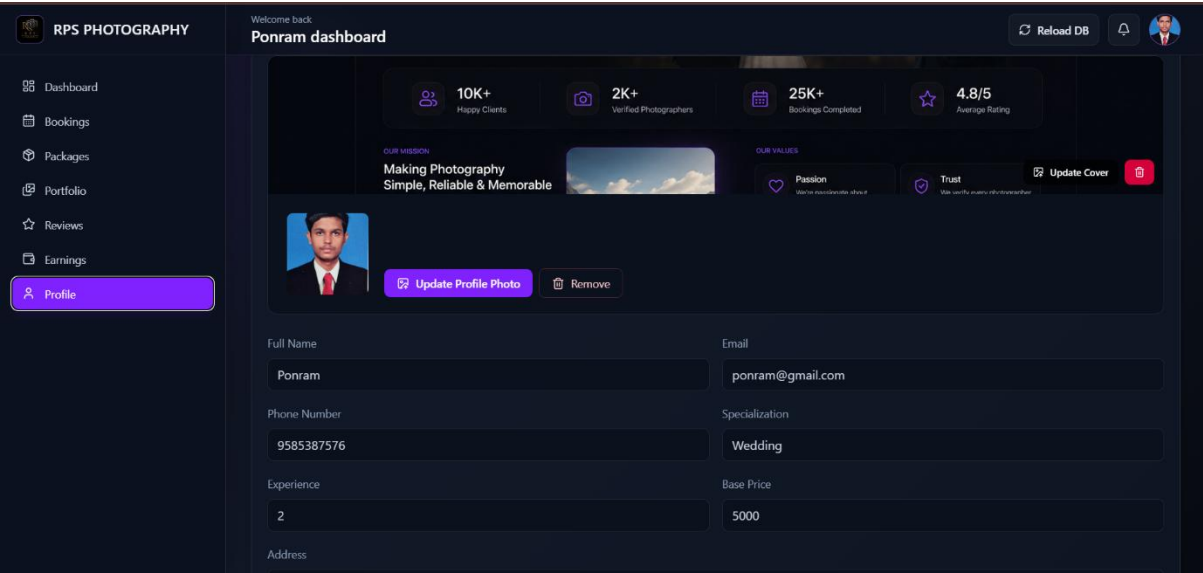


Figure: 5.2.14 Photographer Profile Update Page

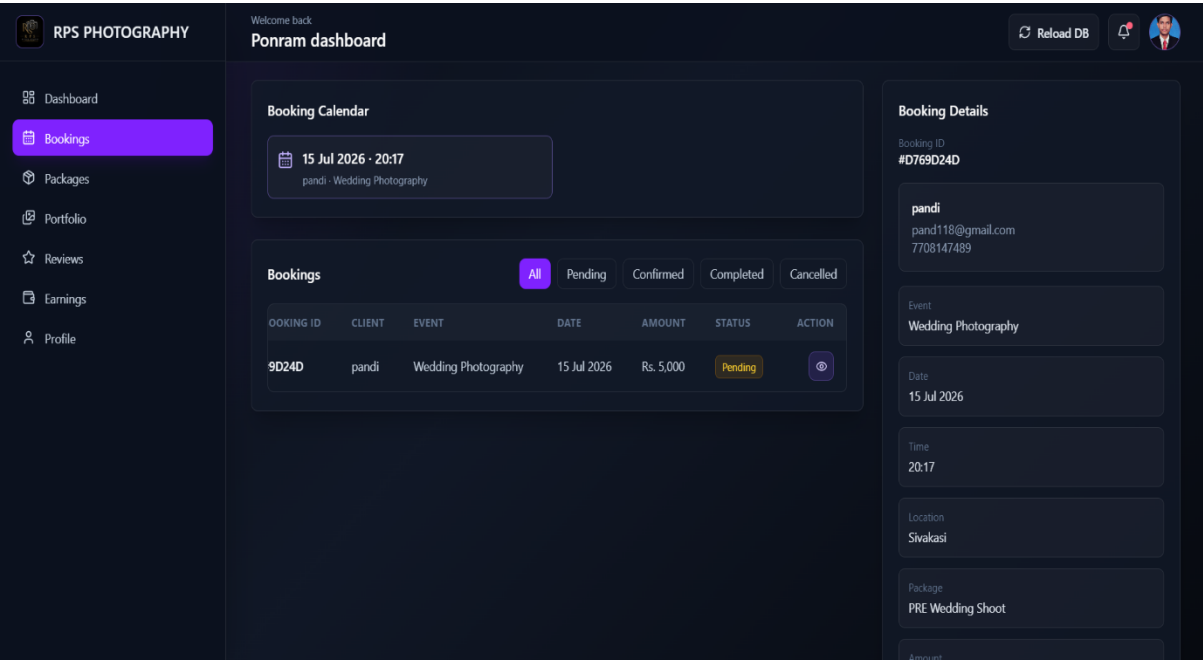
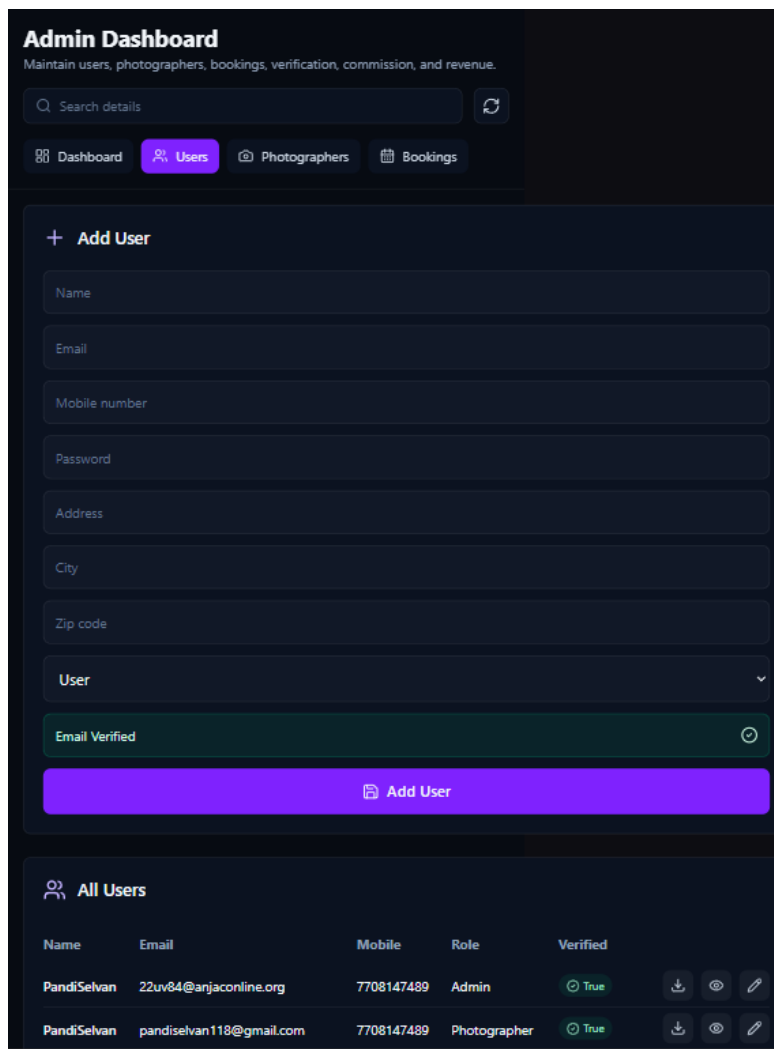


Figure: 5.2.15 Booking Control Page



Admin Dashboard
Maintain users, photographers, bookings, verification, commission, and revenue.

Q Search details

Dashboard Users Photographers Bookings

+ Add User

Name

Email

Mobile number

Password

Address

City

Zip code

User

Email Verified

Add User

All Users

Name	Email	Mobile	Role	Verified	
PandiSelvan	22uv84@anjaonline.org	7708147489	Admin	True	  
PandiSelvan	pandiselvan118@gmail.com	7708147489	Photographer	True	  

Figure: 5.2.16 Admin Add User

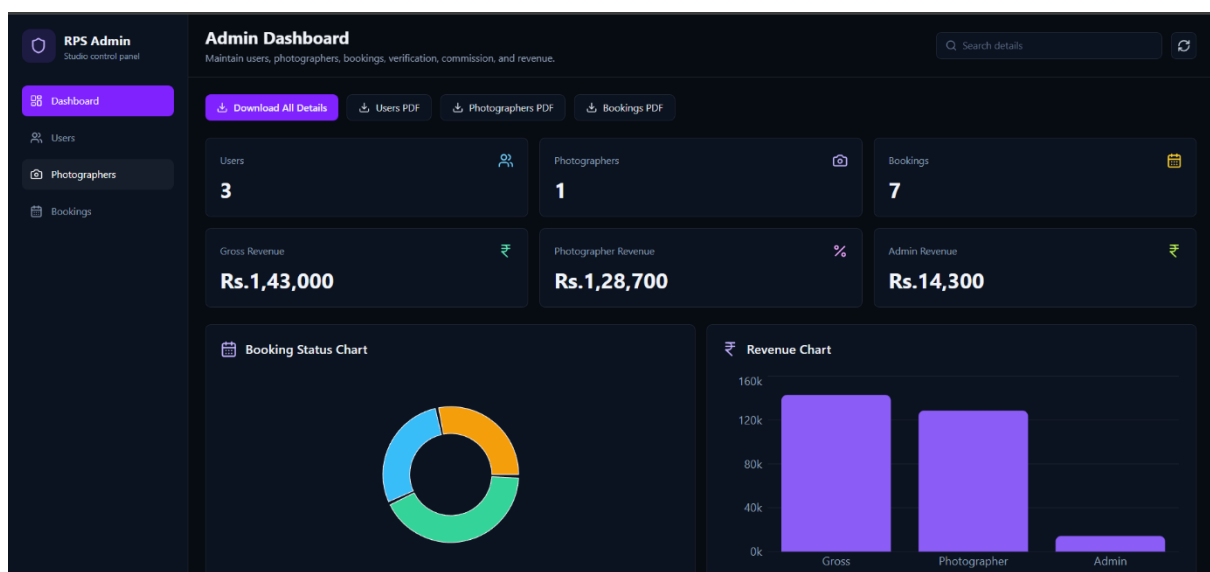


Figure: 5.2.17 Admin Dashboard

6. Conclusion

The **Photographer Booking Web Application** is developed to simplify the process of finding and booking photographers for various events. This system provides an easy platform for users to search photographers, view their profiles, book services, and make secure payments. It also helps photographers manage their profiles, upload portfolios, handle bookings, and maintain their availability.

The administrator can monitor and manage all activities such as user details, photographer details, bookings, and payments efficiently. The application reduces manual work, saves time, and improves communication between users and photographers. Additionally, the event gallery feature allows users to upload and store their event photos securely after the completion of the function. Overall, this project provides a user-friendly, secure, and efficient solution for online photographer booking and management.

BIBLIOGRAPHY

Referred books:

- [I 21] Full Stack Development with React and Node.js., First Edition, BPB Publications, India, 2021.
- [K 22] MERN Stack Web Development., First Edition, Notion Press, India, 2022.
- [S 20] Mastering MongoDB and Express.js., First Edition, Orange Education Pvt Ltd, India, 2020.
- [P 21] Modern JavaScript for Full Stack Development., First Edition, Wiley India Pvt Ltd, India, 2021.
- [R 22] Web Application Development using MERN Stack., First Edition, Khanna Book Publishing, India, 2022.

Referred websites:

- <https://www.mongodb.com/> – Official MongoDB Documentation.
- <https://react.dev/> – Official React Documentation.
- <https://nodejs.org/> – Official Node.js Documentation.
- <https://expressjs.com/> – Official Express.js Documentation.
- <https://www.w3schools.com/> – Web Development Tutorials and References.
- <https://www.geeksforgeeks.org/> – Programming Tutorials and Coding Resources (India).
- <https://www.javatpoint.com/> – Java, MERN Stack, and Database Tutorials (India).
- <https://www.tutorialspoint.com/> – Software Development Learning Resources (India).